GUÍA DEL ALUMNO

Un dato, un dueño. Todos lo pueden leer.

TaskFlow, paso a paso: **Pinia, estado global y del servidor, persistencia, depuración y Tailwind CSS.**

Gestión del estado en Vue.js
Vue 3.5 · Pinia 4 · Tailwind 4

Oscar Alvarado
Frontend Tech Lead

Universidad San Martín de Porres
Lima, Perú

ANTES DE EMPEZAR · QUÉ VAS A ENCONTRAR EN ESTA GUÍA

TaskFlow: una app para todo el módulo

Empezamos **TaskFlow**, un gestor de proyectos y tareas que iremos completando en las cuatro sesiones del módulo: hoy el **estado** (Pinia), luego las **rutas** (Vue Router), la integración con **Firebase** y el **control de acceso**. Cada tema de esta guía responde la pregunta que dejó el anterior.

Las siete preguntas de la teoría

1
¿Cómo le damos estilo?
`Tailwind CSS`

2
¿Por qué un store?
`Pinia`

3
¿Cómo se escribe?
`setup stores`

4
¿Cómo lo uso?
`storeToRefs`

5
¿Dónde va cada dato?
`local · global · servidor`

6
¿Y si viene de la red?
`cargando · error · datos`

7
¿Y si recargo? ¿Cómo lo depuro?
`persistencia · DevTools`

TaskFlow a lo largo del módulo

SESIÓN	QUÉ SE AGREGA A TASKFLOW
1 · **Estado** (hoy)	Una pantalla con stores de tareas, tema y sesión, y datos de un servidor simulado. Estilo con Tailwind.
2 · Vue Router	Páginas: inicio de sesión, proyectos, detalle de un proyecto.
3 · Firebase	El servidor simulado se reemplaza por Firebase; los stores no cambian.
4 · Acceso	Rutas protegidas y permisos por rol (admin / miembro).

Lo que necesitas instalado

- **Node.js 22.18 o superior** (comprueba con `node -v`).
- **VS Code** con las extensiones **Vue (Official)** y **Tailwind CSS IntelliSense**. Opcional: **Prettier**.
- **Chrome** con la extensión **Vue.js devtools** (o usa el plugin de Vite: `Alt+Shift+D`).

TEORÍA · TAILWIND CSS EN 5 IDEAS 4 MIN

Antes del estado, la pantalla

CONCEPTO Tailwind CSS

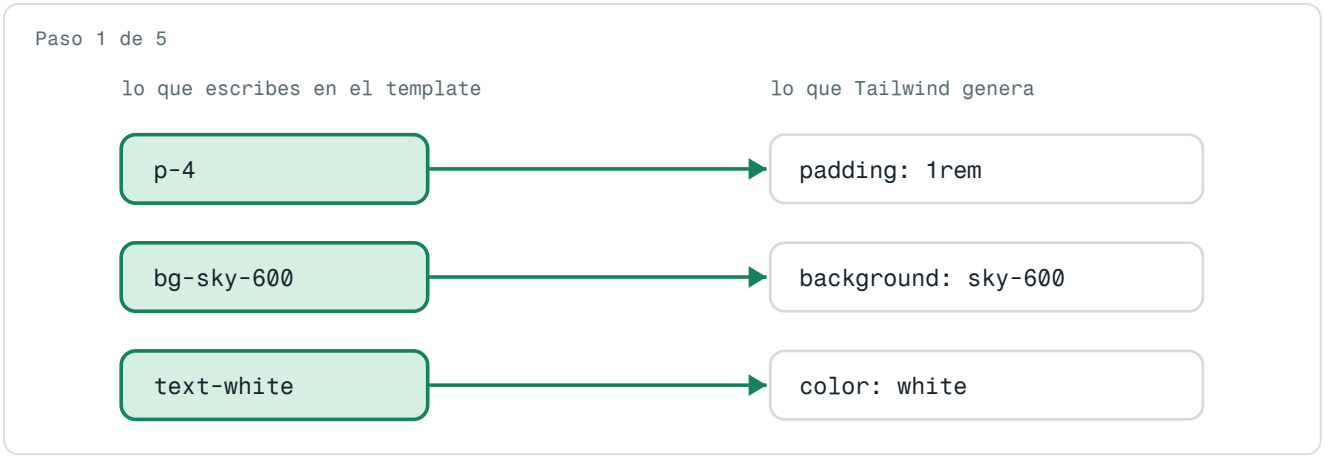
Antes del estado, la pantalla: ¿cómo le damos estilo a TaskFlow sin escribir un CSS aparte?

QUÉ ES

Un framework de **clases utilitarias**: cada clase hace una sola cosa (`p-4` , `bg-sky-600`) y las combinas en el propio template.

PARA QUÉ SE USA

Estilizar sin salir del componente, sin inventar nombres de clases y sin que el CSS crezca con cada pantalla.





Cómo funciona, paso a paso

1. **Una clase, una declaración.** `p-4` es `padding: 1rem` . Tailwind solo genera el CSS de las clases que usas.
2. **Se combinan en el elemento.** Cuatro clases y el botón ya tiene color, texto, espacio y bordes redondeados. Sin tocar un archivo CSS.
3. **Variantes con prefijo.** `hover:` y `focus:` aplican la clase solo en ese estado. El prefijo, dos puntos y la clase.
4. **Responsive y tema.** `md:` aplica desde 768px (lo base es el móvil). `dark:` aplica en tema oscuro.
5. **Si se repite, componente.** Tres tarjetas con las mismas clases se vuelven un `<TareaCard />` : el estilo se escribe una vez.

REGLA

Si una combinación de clases se **repite**, no la copies: extrae un **componente** (`TareaCard`). Las clases viven en un solo lugar.

> **Y entonces...** TaskFlow ya se ve bien. ¿Cómo comparten datos sus componentes?

De «¿por qué?» a «¿cómo lo uso?»

CONCEPTO Pinia · el store

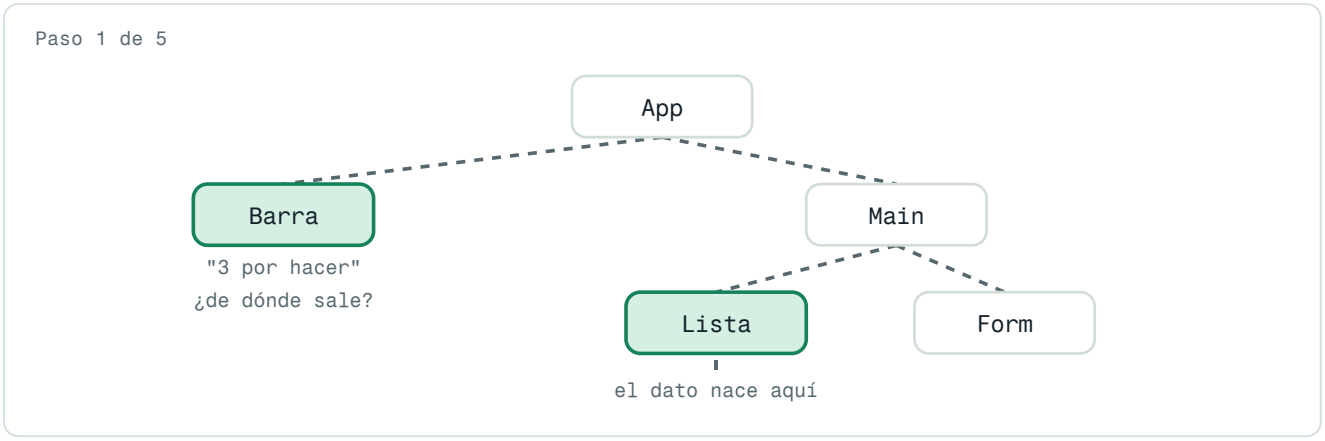
¿Cómo comparten datos dos componentes que no son padre e hijo?

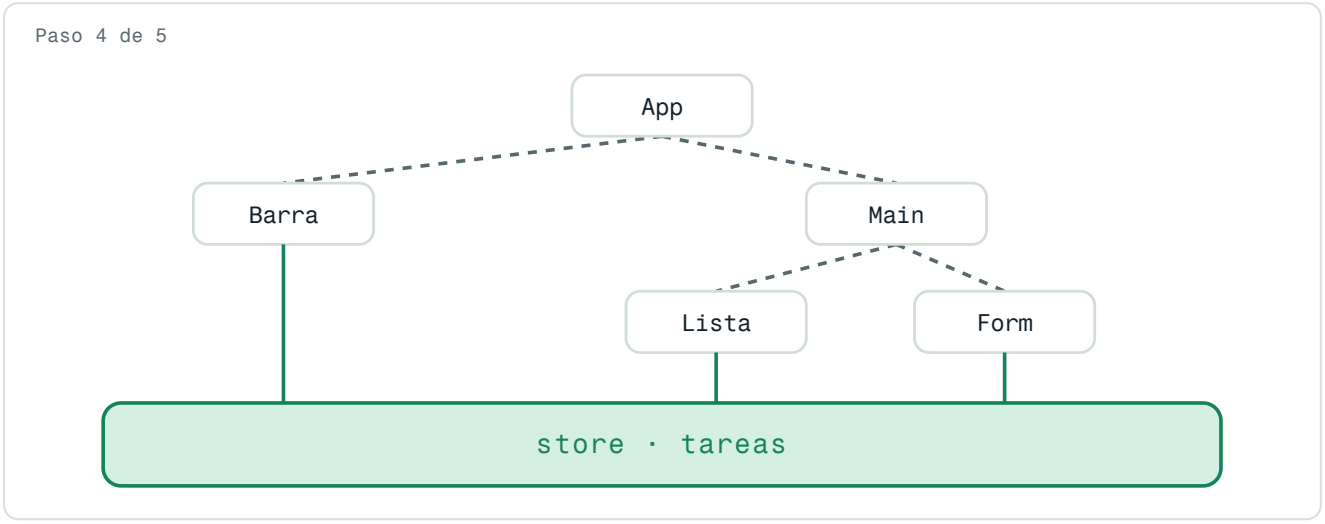
QUÉ ES

La librería **oficial** de estado global de Vue. Un **store** es un objeto reactivo al que cualquier componente accede, sin pasar props.

PARA QUÉ SE USA

Que la cabecera, la lista y el formulario de TaskFlow lean y cambien las mismas tareas sin depender de quién es padre de quién.





Cómo funciona, paso a paso

1. **El problema.** La lista sabe cuántas tareas quedan; la barra superior lo necesita para mostrarlo. No son padre e hijo.
2. **Prop drilling.** El dato sube y baja por `App` y `Main` con props y emits, aunque ellos no lo usan. Cada cambio toca cuatro archivos.
3. **Dos copias.** Si cada rama guarda su versión, tarde o temprano dicen cosas distintas: la barra marca 3 y la lista 4.
4. **Un store.** El dato vive en un solo lugar y cada componente se conecta directo. Nadie en el medio.

5 **El flujo.** Un componente llama una acción, el store cambia su estado y todos los que lo leen se actualizan solos (la reactividad de siempre).

REGLA

Un dato, un dueño. El store guarda el dato; los componentes lo leen y piden cambiarlo con una acción, no guardan su propia copia.

Y entonces... Ya sabemos qué es un store. ¿Cómo se escribe uno?

CONCEPTO defineStore

¿Cómo se escribe un store?

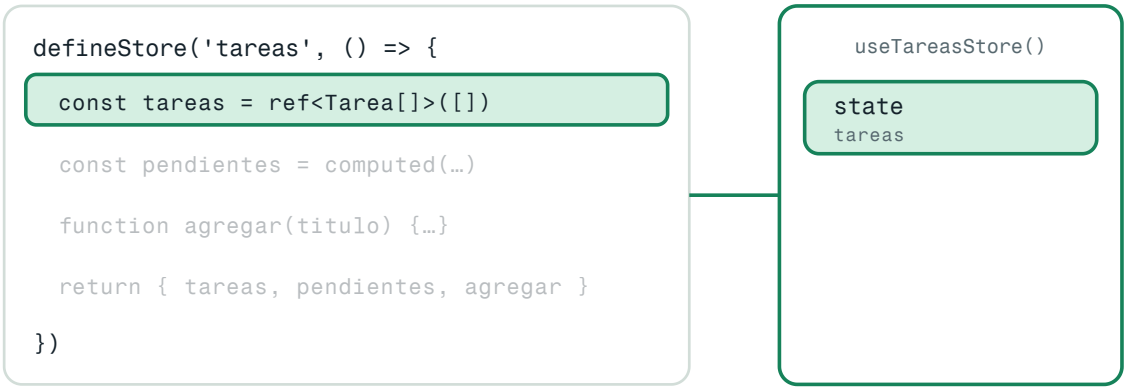
QUÉ ES

Una función `setup` como la de un componente: lo que declaras dentro y retornas es el store (setup store).

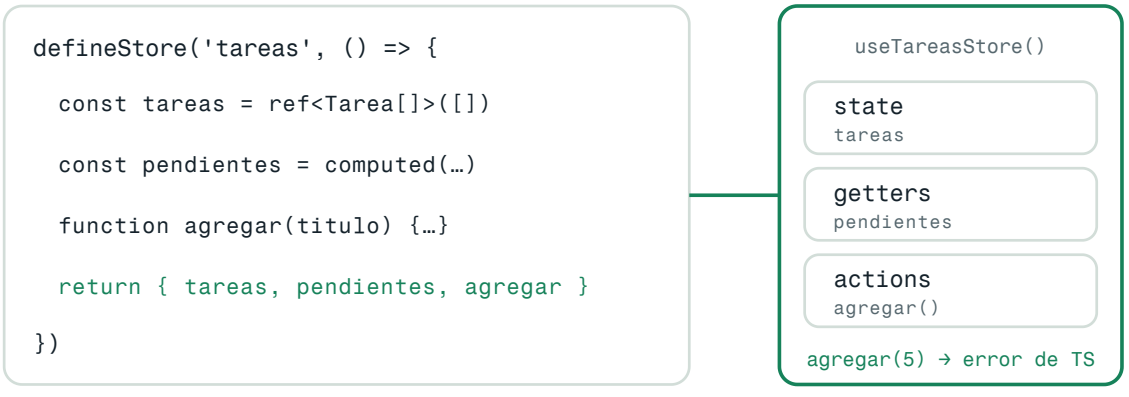
PARA QUÉ SE USA

Reusar lo que ya sabes: `ref` es el **state**, `computed` son los **getters** y las funciones son las **acciones**.

Paso 2 de 5



Paso 5 de 5



Cómo funciona, paso a paso

1 `defineStore('tareas', () => { … })`: un **id único** y una función. La convención es llamarlo `useTareasStore`.

2. **State** = `ref`. Es el dato: `const tareas = ref<Tarea[]>([])`. Se tipa como cualquier `ref`.
3. **Getters** = `computed`. Datos derivados, con caché: `pendientes` se recalcula solo cuando cambian las tareas.
4. **Acciones** = funciones. Cambian el state y pueden ser `async`. Ahí va la lógica que antes estaba repartida en los componentes.
5. El `return` es la API pública. Queda **tipada**: el editor autocompleta `tareas.agregar` y marca `agregar(5)` como error.

REGLA

Lo que quieras usar desde fuera **tiene que estar en el** `return`. Lo que no retornas es privado (y no se ve en las DevTools).

> **Y entonces...** Con el store escrito, ¿cómo lo usa un componente sin romper la reactividad?

CONCEPTO storeToRefs

Tenemos el store: ¿cómo lo usa un componente sin perder la reactividad?

QUÉ ES

`useTareasStore()` devuelve el store ya reactivo. `storeToRefs` saca de él su state y getters como `ref`.

PARA QUÉ SE USA

Usar `pendientes` en el template sin escribir `tareas.pendientes` cada vez, sin perder la conexión.

Paso 2 de 4



copió el 3 de ese momento: la pantalla no se entera

Paso 3 de 4



devuelve refs conectadas al store: state y getters

Cómo funciona, paso a paso

1. **El store completo.** `const tareas = useTareasStore()` y en el template `{{ tareas.pendientes }}`. Siempre lee el valor actual.
2. **La trampa.** `const { pendientes } = tareas` copia el 3 de ese instante. El store pasa a 2 y la pantalla sigue diciendo 3.
3. **La solución.** `const { pendientes } = storeToRefs(tareas)` devuelve refs conectadas al store: la pantalla marca 2.
4. **Las acciones** son funciones, no tienen nada que perder: `const { agregar } = tareas` funciona tal cual.

REGLA

No desestructures el state ni los getters: te llevas una copia congelada. Usa `storeToRefs`. Las **acciones** sí se desestructuran.

> **Y entonces...** Ya sabemos escribir y usar un store. ¿Todo el estado de la app debe ir ahí?

TEORÍA · ORGANIZAR EL ESTADO 14 MIN

Cada dato, en su lugar

CONCEPTO Local · global · servidor

Si Pinia comparte datos, ¿todo el estado va en un store?

QUÉ ES

Tres tipos de estado con tres hogares: **local** (un componente), **global** (la app) y del **servidor** (datos de otro que llegan por red).

PARA QUÉ SE USA

Decidir dónde va cada dato **antes** de escribirlo, para no meterlo todo en Pinia ni dejarlo todo en los componentes.





Cómo funciona, paso a paso

1. **Dos preguntas por dato:** ¿quién lo necesita? y ¿de dónde viene? Las respuestas lo mandan a una de tres columnas.
2. **Local.** Lo usa un solo componente: el texto que escribes en el formulario de nueva tarea. Un `ref` en el componente y listo.
3. **Global.** Lo usan varios componentes lejanos: el **tema**, el **usuario** y el **filtro**. Van a un store de Pinia.
4. **Servidor.** No es tuyo, llega tarde y puede fallar: la **lista de tareas**. Va a un store, pero con estados extra (siguiente slide).

5 **El mapa de TaskFlow.** Esto es lo que vamos a construir: un `ref` local, tres datos globales y una lista que viene del servidor.

REGLA

Empieza local. Sube a un store solo cuando dos componentes que no son padre e hijo lo necesitan. Lo que viene de la red se trata aparte.

Y entonces... El estado del servidor es el más delicado: ¿qué tiene de especial?

CONCEPTO cargando · error · datos

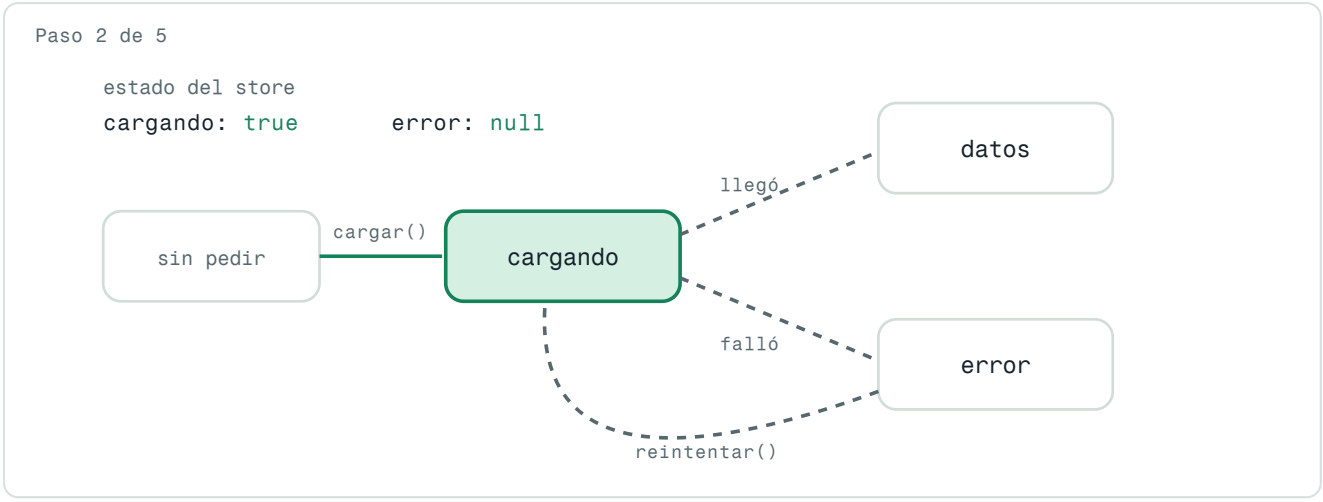
Un dato que viene de la red: ¿qué tiene de especial?

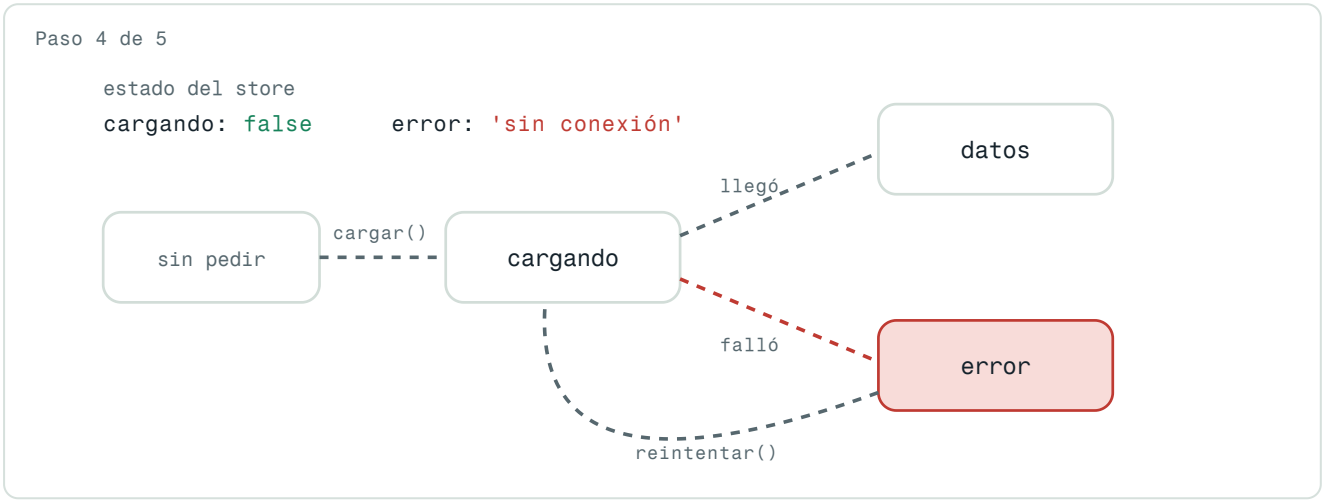
QUÉ ES

Un dato del servidor no es tuyo: tarda en llegar, puede fallar y tu copia se queda vieja. Además del dato, hay que guardar **en qué situación está**.

PARA QUÉ SE USA

Mostrar “Cargando...”, un error con “Reintentar” o la lista, en vez de una pantalla vacía que no explica nada.





Cómo funciona, paso a paso

1 **La verdad está afuera.** El servidor tiene 5 tareas; tu store tiene una copia que tardó en llegar y puede estar vieja o no llegar.

2. **Cargando.** `cargar()` pone `cargando = true` y la pantalla muestra “Cargando…”.
3. **Datos.** La respuesta llega: se guarda en el store, `cargando = false` y se muestra la lista.
4. **Error.** La red falla: `error` guarda el mensaje y la pantalla ofrece **Reintentar**. El usuario nunca ve un lienzo en blanco.
5. **Reintentar.** Vuelve a llamar `cargar()`: limpia el error y regresa a cargando. Es el mismo camino.

REGLA

Todo dato del servidor va con **tres estados juntos**: `cargando`, `error` y el dato. Una acción `cargar()` los maneja con `try / catch / finally`.

> **Y entonces…** Ya tenemos estado global y del servidor. ¿Qué pasa con todo eso si el usuario recarga la página?

CONCEPTO Persistencia

El store vive en memoria: ¿qué pasa si recargo la página?

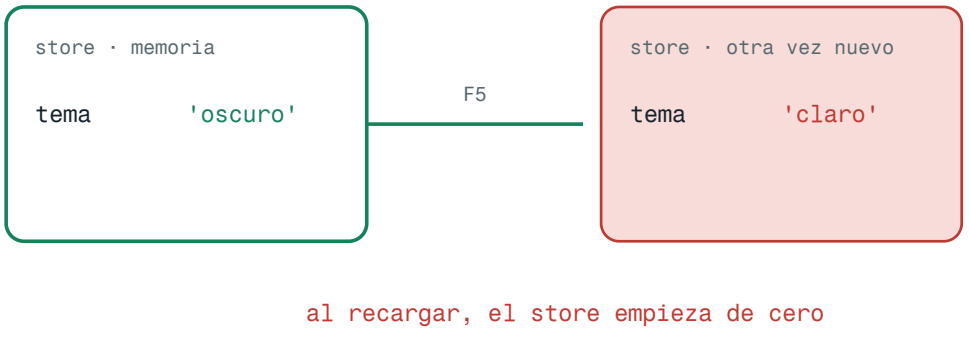
QUÉ ES

Guardar una copia del estado en el navegador (`localStorage`) y leerla al arrancar, para que sobreviva a recargar o cerrar la pestaña.

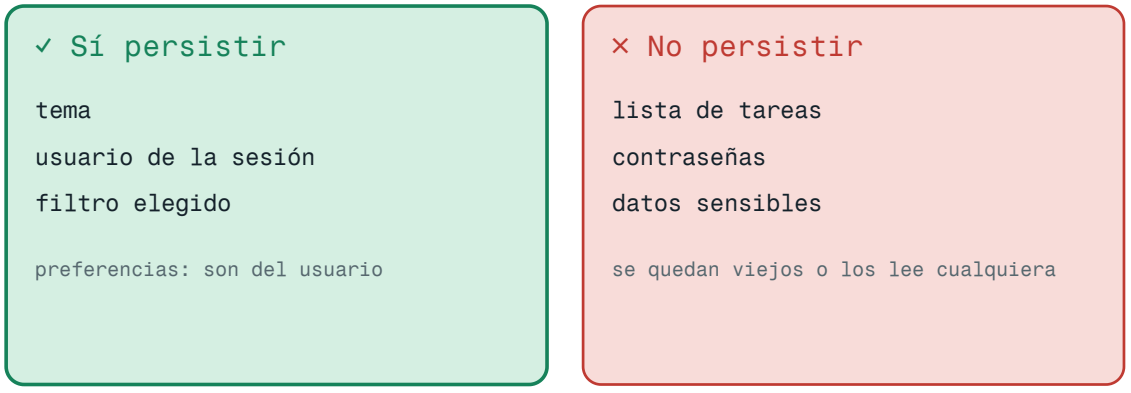
PARA QUÉ SE USA

Recordar el tema oscuro o la sesión del usuario entre visitas, sin que tenga que volver a elegirlos.

Paso 1 de 5



Paso 4 de 5



Cómo funciona, paso a paso

1. **Memoria.** Al recargar (F5) el store se crea de nuevo y vuelve a sus valores iniciales: el tema oscuro se pierde.
2. **Escribir.** Un `watch` dentro del store guarda en `localStorage` cada vez que el tema cambia.
3. **Leer.** Al crear el store, el valor inicial sale de `localStorage`. Así el tema oscuro vuelve solo.
4. **Qué sí y qué no.** Sí: tema, usuario, filtro. No: la lista del servidor ni nada sensible.
5. **El plugin.** `pinia-plugin-persistedstate` hace esto con `{ persist: true }`. Primero lo hacemos a mano para entender qué hace.

REGLA

Persiste **preferencias y sesión**. No persistas datos del servidor (se quedan viejos) ni secretos: `localStorage` lo puede leer cualquier script de la página.

> **Y entonces...** El estado ya sobrevive. Cuando algo falla, ¿cómo miramos dentro del store?

CONCEPTO DevTools · Pinia

Algo no se ve como esperas: ¿cómo miramos dentro del store?

QUÉ ES

La pestaña **Pinia** de las Vue DevTools: lista tus stores con su state y getters, y registra en el Timeline cada acción que se ejecuta.

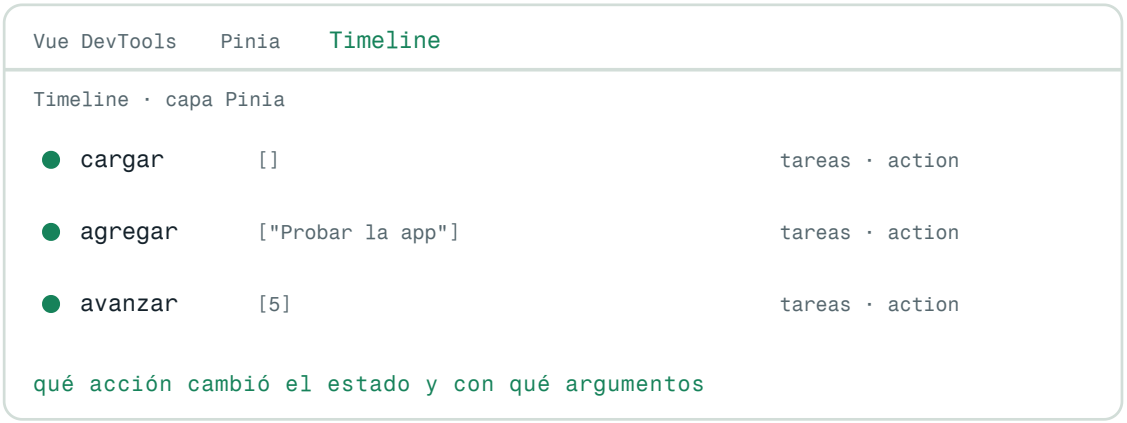
PARA QUÉ SE USA

Saber **qué acción** cambió el estado y **con qué argumentos**, y probar valores sin tocar el código.

Paso 2 de 5



Paso 4 de 5



Cómo funciona, paso a paso

1. **La pestaña Pinia** lista cada store que creaste: `tareas`, `sesion`, `preferencias`. (Las DevTools las viste en el módulo 1.)
2. **Al elegir un store** ves su state y sus getters con los valores de ahora mismo, sin `console.log`.
3. **Se pueden editar.** Cambia un valor del state y la página reacciona: sirve para probar casos difíciles de provocar.
4. **Timeline.** Cada acción queda registrada con su nombre y argumentos, en orden. Aquí aparece quién cambió el estado.
5. **Por código.** `$onAction` rodea cada acción (antes, después, si falla) y `$subscribe` avisa de cada cambio del state.

REGLA

Estado mal: abre el store, mira qué acción corrió en el Timeline y con qué datos. Para registrar desde el código usa `$onAction` y `$subscribe`.

> **Y entonces...** Ya tenemos todas las piezas. Vamos a armarlas en TaskFlow.

CÓDIGO EN VIVO · MOMENTO 0: CREAR, INSTALAR Y CONFIGURAR 8 MIN

Un proyecto nuevo con Pinia y Tailwind

Momento 0 de 5 · Instalar y configurar

· 8 min

1. **Terminal** — crea el proyecto con `--bare` (solo TypeScript). El asistente normal ofrece marcar Pinia y Router; hoy los instalamos a mano para ver qué hace cada pieza.
2. **Terminal y main.ts** — `npm install pinia` y registra Pinia con `app.use(createPinia())` *antes* de `mount`.
3. **Terminal, vite.config.ts, main.css, main.ts** — `npm install tailwindcss @tailwindcss/vite`, agrega el plugin, escribe `@import "tailwindcss"` en el CSS e impórtalo en `main.ts`. Si las clases no se aplican, reinicia `npm run dev`.
4. **types.ts, stores/prueba.ts, TareaCard, App.vue** — prueba las dos cosas: un store de tres líneas y una tarjeta con clases de Tailwind. Pasa el mouse por los botones (`hover:`) y encoge la ventana.
5. **Terminal** — ⏭ `npm install -D prettier prettier-plugin-tailwindcss`: ordena las clases al guardar.

terminal

```
npm create vue@latest taskflow -- --ts --bare
cd taskflow
npm install
npm install pinia
npm install tailwindcss @tailwindcss/vite
npm run dev
```

vite.config.ts

```ts
import { fileURLToPath, URL } from 'node:url'

import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import vueDevTools from 'vite-plugin-vue-devtools'
import tailwindcss from '@tailwindcss/vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [vue(), vueDevTools(), tailwindcss()],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url)),
    },
  },
})
```

src/assets/main.css

```
@import 'tailwindcss';

/* El tema oscuro se activa con la clase "dark" en <html>, no con el sistema operativo */
@custom-variant dark (&:where(.dark, .dark *));
```

src/main.ts

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import './assets/main.css'

const pinia = createPinia()

createApp(App).use(pinia).mount('#app')
```

src/components/TareaCard.vue

```
<script setup lang="ts">
import type { Estado, Tarea } from '../types'

defineProps<{ tarea: Tarea; puedeEliminar: boolean }>()
defineEmits<{ avanzar: [id: number]; eliminar: [id: number] }>()

const etiquetas: Record<Estado, string> = {
  pendiente: 'Pendiente',
  'en-curso': 'En curso',
  hecha: 'Hecha',
}
const colores: Record<Estado, string> = {
  pendiente: 'bg-slate-100 text-slate-700 dark:bg-slate-700 dark:text-slate-200',
  'en-curso': 'bg-amber-100 text-amber-800 dark:bg-amber-900 dark:text-amber-100',
  hecha: 'bg-emerald-100 text-emerald-800 dark:bg-emerald-900 dark:text-emerald-100',
}
</script>

<template>
  <li
    class="flex flex-col gap-3 rounded-lg border border-slate-200 bg-white p-4 shadow-sm
   transition hover:shadow-md sm:flex-row sm:items-center sm:justify-between dark:border-slate-700
   dark:bg-slate-800"
  >
    <div class="flex items-center gap-3">
      <span class="rounded-full px-2.5 py-0.5 text-xs font-medium" :class="colores[tarea.estado]">
        {{ etiquetas[tarea.estado] }}
      </span>
      <span :class="tarea.estado === 'hecha' && 'text-slate-400 line-through'">
        {{ tarea.titulo }}
      </span>
    </div>
    <div class="flex gap-2">
      <button
        v-if="tarea.estado !== 'hecha'"
        type="button"
        class="rounded bg-sky-600 px-3 py-1 text-sm text-white hover:bg-sky-700"
        @click="$emit('avanzar', tarea.id)"
      >
```

```
          Avanzar
        </button>
        <button
          v-if="puedeEliminar"
          type="button"
          class="rounded border border-slate-300 px-3 py-1 text-sm text-slate-600 hover:bg-slate-100
    dark:border-slate-600 dark:text-slate-300 dark:hover:bg-slate-700"
          @click="$emit('eliminar', tarea.id)"
        >
          Eliminar
        </button>
      </div>
    </li>
  </template>
```

CÓDIGO EN VIVO · MOMENTO 1: EL STORE DE TAREAS 10 MIN

State, getters y acciones tipadas

Momento 1 de 5 · El store de tareas

· 10 min

1. **stores/tareas.ts** — escríbelo en cuatro vueltas: **state** (`tareas`, `filtro`), **getters** (`visibles`, `pendientes`), **acciones** y el `return`. Lo que no se retorna no existe para fuera.
2. **App.vue** — usa el store. Primero `const { pendientes } = tareas` y pulsa *Avanzar*: el contador no baja (la trampa). Luego `storeToRefs(tareas)`.
3. **NuevaTarea, FiltroTareas** — el título del formulario es un `ref` **local**; el filtro vive en el store porque lo usan dos componentes.

src/stores/tareas.ts · al final del Momento 1

```
import { computed, ref } from 'vue'
import { defineStore } from 'pinia'
import { crearTarea, fetchTareas } from '../services/tareasApi'
import type { Estado, Filtro, Tarea } from '../types'

const SIGUIENTE: Record<Estado, Estado> = {
  pendiente: 'en-curso',
  'en-curso': 'hecha',
  hecha: 'hecha',
}

export const useTareasStore = defineStore('tareas', () => {
  // state
  const tareas = ref<Tarea[]>([])
  const cargando = ref(false)
  const error = ref<string | null>(null)
  const filtro = ref<Filtro>('todas')

  // getters
  const visibles = computed(() =>
    filtro.value === 'todas' ? tareas.value : tareas.value.filter((t) => t.estado ===
   filtro.value),
  )
  const pendientes = computed(() => tareas.value.filter((t) => t.estado !== 'hecha').length)

  // actions
  async function cargar() {
    cargando.value = true
    error.value = null
    try {
      tareas.value = await fetchTareas()
    } catch (e) {
      error.value = e instanceof Error ? e.message : 'Error desconocido'
    } finally {
      cargando.value = false
    }
  }
  async function agregar(titulo: string) {
    try {
      tareas.value.push(await crearTarea(titulo))
    } catch (e) {
      error.value = e instanceof Error ? e.message : 'Error desconocido'
    }
  }
```

```
  function avanzar(id: number) {
    const tarea = tareas.value.find((t) => t.id === id)
    if (tarea) tarea.estado = SIGUIENTE[tarea.estado]
  }
  function eliminar(id: number) {
    tareas.value = tareas.value.filter((t) => t.id !== id)
  }

  return {
    tareas,
    cargando,
    error,
    filtro,
    visibles,
    pendientes,
    cargar,
    agregar,
    avanzar,
    eliminar,
  }
})
```

CÓDIGO EN VIVO · MOMENTO 2: TEMA Y SESIÓN PERSISTIDOS 8 MIN

Que el estado sobreviva a recargar

Momento 2 de 5 · Tema y sesión persistidos

· 8 min

1. **stores/preferencias.ts** — al crear el store se **lee** `localStorage`; un `watch` lo **guarda** y aplica la clase `dark` a `<html>`.
2. **assets/main.css** — `@custom-variant dark`: en Tailwind 4, `dark:` depende de esa clase y no del sistema operativo.
3. **stores/sesion.ts** — ⚡ el usuario con su `rol` es estado **global**. Se persiste el usuario, nunca una contraseña.
4. **BarraSuperior, TareaCard, App.vue** — entra como Ana (admin): aparece «Eliminar». Como Luis (miembro): no. Cambia a oscuro y **recarga**: todo sigue ahí.

OJO

Ocultar un botón **no es seguridad**: quien conozca la acción puede llamarla. El control de acceso de verdad se ve en la sesión 4.

src/stores/preferencias.ts

```ts
import { ref, watch } from 'vue'
import { defineStore } from 'pinia'
import { guardar, leer } from '../lib/almacenamiento'

export type Tema = 'claro' | 'oscuro'

const CLAVE = 'taskflow:tema'

export const usePreferenciasStore = defineStore('preferencias', () => {
  // Al crear el store se lee lo guardado: así sobrevive a recargar la página
  const tema = ref<Tema>(leer<Tema>(CLAVE, 'claro'))

  function alternarTema() {
    tema.value = tema.value === 'claro' ? 'oscuro' : 'claro'
  }

  // Cada cambio se guarda y se aplica a <html>
  watch(
    tema,
    (valor) => {
      guardar(CLAVE, valor)
      document.documentElement.classList.toggle('dark', valor === 'oscuro')
    },
    { immediate: true },
  )

  return { tema, alternarTema }
})
```

src/stores/sesion.ts

```ts
import { computed, ref, watch } from 'vue'
import { defineStore } from 'pinia'
import { borrar, guardar, leer } from '../lib/almacenamiento'
import type { Usuario } from '../types'

const CLAVE = 'taskflow:sesion'

/** Usuarios de la demo: en la sesión 3 vendrán de Firebase Auth */
export const USUARIOS_DEMO: Usuario[] = [
  { id: 1, nombre: 'Ana', rol: 'admin' },
  { id: 2, nombre: 'Luis', rol: 'miembro' },
]

export const useSesionStore = defineStore('sesion', () => {
  const usuario = ref<Usuario | null>(leer<Usuario | null>(CLAVE, null))

  const autenticado = computed(() => usuario.value !== null)
  const esAdmin = computed(() => usuario.value?.rol === 'admin')

  function entrar(id: number) {
    usuario.value = USUARIOS_DEMO.find((u) => u.id === id) ?? null
  }
  function salir() {
    usuario.value = null
  }

  watch(usuario, (valor) => (valor ? guardar(CLAVE, valor) : borrar(CLAVE)))

  return { usuario, autenticado, esAdmin, entrar, salir }
})
```

CÓDIGO EN VIVO · MOMENTO 3: DATOS DEL SERVIDOR 7 MIN

Cargando, error y reintentar

Momento 3 de 5 · Datos del servidor

· 7 min

1. **services/tareasApi.ts** — ⚡ un servidor simulado: funciones `async` que tardan 800 ms y pueden fallar. En la sesión 3 se cambia este archivo por Firebase y los stores no se tocan.
2. **stores/tareas.ts** — agrega `cargando`, `error` y `cargar()` con `try / catch / finally`. La lista empieza vacía.
3. **App.vue** — tres caras con `v-if / v-else-if / v-else`: «Cargando…», el error con *Reintentar*, y la lista.
4. **Navegador** — marca *Simular error de red*, agrega una tarea y verás el error; desmarca y pulsa *Reintentar*.

src/services/tareasApi.ts

```
import { reactive } from 'vue'
import type { Tarea } from '../types'

/**
 * Servidor simulado: responde con retraso y, si se pide, con error.
 * En la sesión 3 este archivo se reemplaza por Firestore; los stores no cambian.
 */

/** Interruptor de la demo: la clase lo usa para provocar un error de red */
export const red = reactive({ falla: false })

const LATENCIA_MS = 800

// La "base de datos" vive aquí, no en el navegador del usuario
const baseDeDatos: Tarea[] = [
  { id: 1, proyectoId: 1, titulo: 'Definir el alcance del proyecto', estado: 'hecha' },
  { id: 2, proyectoId: 1, titulo: 'Diseñar la pantalla principal', estado: 'en-curso' },
  { id: 3, proyectoId: 1, titulo: 'Crear el store de tareas', estado: 'pendiente' },
  { id: 4, proyectoId: 1, titulo: 'Probar la app en el móvil', estado: 'pendiente' },
]
let siguienteId = baseDeDatos.length + 1

const esperar = (ms: number) => new Promise((resolver) => setTimeout(resolver, ms))

async function conectar() {
  await esperar(LATENCIA_MS)
  if (red.falla) throw new Error('No se pudo conectar con el servidor')
}

export async function fetchTareas(): Promise<Tarea[]> {
  await conectar()
  return baseDeDatos.map((t) => ({ ...t }))
}

export async function crearTarea(titulo: string): Promise<Tarea> {
  await conectar()
  const nueva: Tarea = { id: siguienteId++, proyectoId: 1, titulo, estado: 'pendiente' }
  baseDeDatos.push(nueva)
  return { ...nueva }
}
```

CÓDIGO EN VIVO · MOMENTO 4: DEPURAR 4 MIN

Mirar dentro del store

Momento 4 de 5 · Depurar

▰▰▰▰▰ · 4 min

1. **DevTools · Pinia** — `Alt+Shift+D`: elige `tareas` y mira su state y sus getters; edita `filtro` y la lista cambia.
2. **DevTools · Timeline** — cada acción queda registrada con su nombre y argumentos.
3. **App.vue** — ⚡ `tareas.$onAction` para registrar cada acción en la consola (*antes*, *después*, *si falla*).

LA APP, TERMINADA · CHECKPOINT FINAL

TaskFlow al final de la sesión

La pantalla completa: lista con filtro, tema y sesión que sobreviven a recargar, datos de un servidor simulado y un registro de acciones en la consola.

src/App.vue

```
<script setup lang="ts">
import { red } from './services/tareasApi'
import { useSesionStore } from './stores/sesion'
import { useTareasStore } from './stores/tareas'
import BarraSuperior from './components/BarraSuperior.vue'
import TareaCard from './components/TareaCard.vue'
import NuevaTarea from './components/NuevaTarea.vue'
import FiltroTareas from './components/FiltroTareas.vue'

const tareas = useTareasStore()
const sesion = useSesionStore()

tareas.cargar()

// Un registro de todo lo que hacen las acciones: nombre, argumentos y si terminaron bien o mal
tareas.$onAction(({ name, args, after, onError }) => {
  console.log(`▶ ${name}`, args)
  after(() => console.log(`✔️ ${name}`))
  onError((e) => console.warn(`✖️ ${name}`, e))
})
</script>

<template>
  <div class="min-h-screen bg-slate-50 text-slate-800 dark:bg-slate-900 dark:text-slate-100">
    <BarraSuperior />

    <main class="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <label class="flex items-center gap-2 text-sm text-slate-500">
        <input v-model="red.falla" type="checkbox" />
        Simular error de red
      </label>

      <NuevaTarea />
      <FiltroTareas />

      <p v-if="tareas.cargando" class="animate-pulse text-center text-slate-500">
        Cargando tareas…
      </p>
      <div
        v-else-if="tareas.error"
        class="flex items-center justify-between gap-3 rounded border border-red-300 bg-red-50 px-
   4 py-3 text-red-800 dark:border-red-800 dark:bg-red-950 dark:text-red-200"
      >
        <span>{{ tareas.error }}</span>
        <button
          type="button"
          class="rounded bg-red-600 px-3 py-1 text-sm text-white hover:bg-red-700"
          @click="tareas.cargar()"
        >
          Reintentar
        </button>
      </div>
      <template v-else>
```

```
        <ul class="flex flex-col gap-3">
          <TareaCard
            v-for="t in tareas.visibles"
            :key="t.id"
            :tarea="t"
            :puede-eliminar="sesion.esAdmin"
            @avanzar="tareas.avanzar"
            @eliminar="tareas.eliminar"
          />
        </ul>
        <p v-if="tareas.visibles.length === 0" class="text-center text-slate-500">
          No hay tareas en esta vista.
        </p>
      </template>
    </main>
  </div>
</template>
```

CIERRE · PARA SEGUIR

Ocho preguntas, ocho respuestas

PREGUNTA	RESPUESTA
¿Cómo le damos estilo?	clases de Tailwind · componente si se repite
¿Cómo comparten datos componentes lejanos?	un store de Pinia
¿Cómo se escribe un store?	`defineStore` con `ref`, `computed` y funciones
¿Cómo lo uso sin perder reactividad?	`storeToRefs` (las acciones, directo)
¿Dónde va cada dato?	local · global · servidor
¿Y los datos de la red?	cargando + error + datos
¿Y si recargo?	persistir preferencias y sesión (no datos del servidor ni secretos)
¿Cómo depuro un store?	pestaña Pinia · `$onAction`

Para practicar

Agrega a `useTareasStore` un getter `hechas` y una acción `limpiarHechas()` con su botón, visible solo para el admin. Bonus: persiste el `filtro` y compáralo con `pinia-plugin-persistedstate`.

Glosario

TÉRMINO	EN UNA FRASE
Store	Objeto reactivo compartido por toda la app; guarda un dato y la lógica para cambiarlo.
Setup store	Store escrito como una función `setup`: `ref` = state, `computed` = getters, funciones = acciones.
State · getter · acción	El dato; un dato derivado con caché; una función que cambia el dato.
`storeToRefs`	Saca el state y los getters de un store como `ref` sin perder la reactividad.
Estado local · global · del servidor	De un componente; de toda la app; de datos que pertenecen a otro y llegan por red.
Persistir	Guardar una copia del estado en el navegador para recuperarla al recargar.
`$onAction` · `$subscribe`	Ganchos de un store: rodean cada acción; avisan de cada cambio del state.
Clase utilitaria	Clase de Tailwind que hace una sola cosa (`p-4`, `bg-sky-600`).

Para repasar

Pinia · documentación	pinia.vuejs.org
Setup stores	pinia.vuejs.org
Gestión de estado	vuejs.org
Tailwind CSS · documentación	tailwindcss.com
Tailwind con Vite	tailwindcss.com
pinia-plugin-persistedstate	prazdevs.github.io